SOFTWARE REQUIREMENTS SPECIFICATION

GK Interior — LC-Based Import, Inventory & POS SaaS Platform

Prepared for: GK Interior (Pilot Tenant) — Multi-Tenant SaaS Product

Prepared by: Nexoda

Target Build Agent: Claude Code (MERN Stack + MySQL)

Version: 1.0

Date: September 14, 2026

This document is written for direct implementation by an AI coding agent (Claude Code). Every functional module includes explicit field lists, business rules, calculations, and acceptance criteria so it can be built without further clarificatio

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for a multi-tenant Point-of-Sale (POS), Inventory, Import/LC Management, and business-finance SaaS platform. The first tenant and reference client is GK Interior, a Bangladeshi importer and retailer of aluminium, glass, and home-decor items sourced from China. This document is written specifically to be consumed by an AI coding agent (Claude Code) as the primary build specification. It defines entities, fields, business rules, calculations, workflows, and acceptance criteria in enough detail that the system can be implemented with minimal additional clarification.

1.2 Scope

The system to be built is a web-based, multi-tenant SaaS application named (working title) "ImportPOS" that allows any company with an import → inventory → sales business model to:

- Register as a company (tenant) using only an email address, and manage its own isolated workspace.
- Manage suppliers, Letters of Credit (LC), purchase orders, and all import-related landed costs.
- Automatically distribute LC-related costs (LC opening cost, freight, port cost, CNF cost, transport cost, unloading cost, delivery cost, customs duty, and other charges) across purchased products by weight or by piece quantity, to compute an accurate per-unit landed cost in BDT (Taka) from a foreign-currency (RMB/USD) purchase price.
- Maintain inventory with product variations, multi-LC batch tracking, and branch-wise stock.
- Record sales via cash and credit invoices, print delivery challans, and record money receipts.
- Manage customers (CRM) including credit limits, opening balances, ledgers, and outstanding balances.
- Track daily business expenses, including a dedicated LC-linked delivery-charge entry workflow.
- Manage multiple branches and employees per branch with granular, role-based access control.
- Generate operational and financial reports: purchase, sales, profit/loss, inventory, branch-wise inventory, salesman-wise sales, returns, delivery charges, top-selling items, and more.
- Operate as a subscription-based SaaS product with plan management, billing, and self-service company onboarding.

Out of scope for v1.0: mobile native apps (a responsive web app is in scope), direct integration with Bangladesh Bank LC systems or payment gateways (manual data entry is assumed), and automated customs/tax filing. These may be considered in a future phase.

1.3 Intended Audience

- Claude Code (or any AI/human engineering team) implementing the system from this document.
- Texon's project lead (Ashraful) as the vendor and technical decision-maker.
- GK Interior's management as the pilot client and primary source of business rules.
- Future SaaS customers with similar import-to-retail business models.

1.4 Definitions, Acronyms & Abbreviations

Term	Definition
LC	Letter of Credit — a bank-issued payment guarantee used to import goods from an overseas supplier.
CNF cost	Cost & Freight — shipping-related cost component of an import shipment.
Challan	A delivery note/slip printed and handed to the delivery person / customer, separate from the sales invoice.
Money Receipt	A document/record confirming a customer payment, cash or against a credit invoice.
Landed Cost	The final, all-in cost per unit of a product after distributing all import-related charges across the shipment.
Tenant / Company	A single subscribing business using its own isolated workspace within the SaaS platform.
RBAC	Role-Based Access Control.
POS	Point of Sale.
CRM	Customer Relationship Management.
SKU	Stock Keeping Unit — a unique product/variation identifier.
BDT / Taka	Bangladeshi currency, the base currency for sales and reporting.
RMB / USD	Chinese Yuan / US Dollar — foreign currencies used for supplier purchase transactions.
JWT	JSON Web Token, used for stateless authentication.
MERN	MongoDB, Express, React, Node — referenced stack name; this project substitutes MySQL for the database layer (see §2.4).

1.5 Document Conventions

Requirement identifiers use the format MOD-##, where MOD is a 2–4 letter module code (e.g., LC-01, INV-04, SLS-12). The keywords MUST, SHOULD, and MAY follow standard RFC-2119-style meaning: MUST is mandatory for v1.0, SHOULD is a strong recommendation, and MAY is optional/future.

1.6 Numbering & Language Note

This document formalizes and expands GK Interior's original handwritten process notes (cost items, LC workflow, invoice types, and reporting list) into structured, buildable requirements. Where the client's shorthand was ambiguous, the most operationally sensible interpretation is specified explicitly as the v1.0 behavior, with configuration options noted where relevant.

2. Overall Description

2.1 Product Perspective

The product is a new, standalone, multi-tenant SaaS web application — not an extension of an existing system. Each subscribing company ("tenant") operates in a fully data-isolated workspace with its own branches, employees, suppliers, LCs, products, customers, and financial records. A separate Super Admin (platform owner) layer manages tenant onboarding, subscription plans, billing, and platform-wide monitoring.

2.2 Business Process Context — GK Interior's Import-to-Sale Workflow

The functional design is driven by the following real business process, which the system must fully support end-to-end:

1. Supplier Selection: GK Interior identifies and fixes a supplier in China for aluminium, glass, or home-decor products.
2. LC Opening: GK Interior opens a Letter of Credit through a Bangladeshi bank to guarantee payment to the supplier. The LC has a number, issuing bank, value, currency, and associated cost (LC opening cost/charge).
3. Order Placement: A purchase order is placed against the supplier, referencing the LC, listing products and quantities and their agreed unit price in RMB or USD.
4. Supplier Delivery & Shipping: The supplier manufactures/ships the goods. Freight, port cost, CNF cost, and transport cost are incurred to bring the shipment to Bangladesh.
5. Arrival & Cost Finalization: When the shipment (against a given LC) arrives in Bangladesh, GK Interior records the actual RMB/USD-to-BDT exchange rate used, and finalizes all additional charges: LC opening cost, freight, port cost, CNF cost, transport cost, unloading cost, delivery cost, customs duty, and any other charges.
6. Cost Distribution (Landed Cost Calculation): The total of these additional charges is distributed proportionally across every product unit in that LC shipment, using either weight-based or quantity(piece)-based distribution (selectable per LC/shipment), to produce a final per-unit landed cost in BDT.
7. Inventory Entry: Each product (with its variations, e.g., 3 aluminium profile variations with different prices) is entered into inventory at its computed landed cost, tagged to its originating LC/batch.
8. Sales: Products are sold for cash or on credit. Two invoice types exist (cash invoice, credit invoice), and a separate delivery challan is printed for the delivery person.
9. Post-Sale Operations: Returns, invoice cancellations, credit notes, order cancellations, and money receipts against credit invoices are all tracked.
10. Reporting & Closing: Daily cash closing, daily/monthly profit & loss, and a wide range of operational reports are generated for management.

2.3 User Classes and Characteristics

Role	Description	Typical Permissions
Super Admin (Platform Owner)	Texon / SaaS operator staff. Not tied to any single tenant.	Create/suspend tenants, manage subscription plans & billing, view platform health, impersonate-for-support (audited), manage global settings.

Role	Description	Typical Permissions
Company Owner / Admin	The tenant's top-level account, created at signup.	Full access within their own company: branches, employees, roles, all modules, all reports, subscription management for their company.
Branch Manager	Manages one or more assigned branches.	Full operational access (LC, inventory, sales, expenses, reports) scoped to their assigned branch(es).
Sales Staff / Salesman	Front-desk / counter staff issuing invoices.	Create cash/credit invoices, view stock availability, view own sales performance; no access to costs, LC data, or other branches.
Inventory / Store Manager	Manages stock receipt and warehouse operations.	Receive purchased goods into inventory, manage stock adjustments, view stock reports; no financial/LC editing rights.
Accountant / Finance Officer	Manages expenses, cash closing, and financial reports.	Full access to expense entry, daily cash closing, P&L, LC cost entry; restricted from user/role management.
Delivery Coordinator	Manages delivery challans and delivery-charge entries.	Create/print challans, record delivery charges against LC numbers, update delivery status.

2.4 Operating Environment & Technology Stack

Layer	Technology	Notes
Frontend	React (Vite), Redux Toolkit or React Query, TailwindCSS + a component library (e.g., shadcn/ui)	Responsive, desktop-first with tablet support for counter/POS screens.
Backend / API	Node.js + Express.js (REST API), JWT-based auth	Modular MVC/service-repository structure; versioned API (/api/v1/...).
Database	MySQL (relational)	Chosen over MongoDB for this project because LC costing, inventory ledgers, and financial reports are inherently relational and require transactional integrity (ACID). Use Sequelize or Prisma as ORM.
Authentication	JWT (access + refresh tokens), bcrypt/argon2 password hashing	Optional email OTP for company signup verification.
File/Asset Storage	Cloud object storage (e.g., S3-compatible) for product images, LC/document attachments	Not stored in MySQL as BLOBs.
Background Jobs	Node cron / BullMQ + Redis	For subscription renewal checks, scheduled reports, low-stock alerts, email notifications.
Hosting	Cloud VM or container platform (e.g., AWS/DigitalOcean) with CI/CD	Environment-separated: development, staging, production.
Payments/Subscription Billing	Pluggable payment gateway integration (e.g., SSLCommerz/bKash for BD, or Stripe for international)	Abstracted behind a billing service interface so gateways can be swapped.

2.5 Design and Implementation Constraints

- The system will be built using the MERN-family stack with MySQL as the relational database
- The system MUST be multi-tenant from day one; tenant data isolation is a hard security requirement, not a future enhancement.
- All monetary values MUST be stored with explicit currency and, where foreign-currency purchases are involved, with the exchange rate used and the resulting BDT value — never store only a converted number without retaining the source values.
- All cost-distribution and landed-cost calculations MUST be reproducible/auditable: the system must be able to show exactly how a per-unit cost was derived from an LC's total charges.
- The UI MUST be optimized for fast, low-click daily use at a sales counter (invoice creation) as well as detailed back-office data entry (LC, expenses).

2.6 Assumptions and Dependencies

- Exchange rates (RMB/USD to BDT) are entered manually per LC/shipment by an authorized user; no live FX API is required for v1.0 (may be added later as an enhancement).
- Bank LC issuance itself happens outside the system (at the bank); the system only records LC data and costs, it does not communicate with the bank.
- Each tenant is a single legal company that may operate multiple physical branches under one subscription.
- Users have basic literacy in English and/or Bengali; the UI should support both where feasible (see §6.4).

3. System Architecture Overview

3.1 Multi-Tenancy Model

Approach: Shared database, shared schema, tenant-scoped by company_id (a "pool" model). Every tenant-owned table includes a company_id foreign key, and every query MUST be scoped by the authenticated user's company_id at the application/service layer (never trust a client-supplied company_id). This is simpler to operate and scale for a v1.0 SaaS than database-per-tenant, while still allowing a later migration path to schema-per-tenant for large enterprise customers if needed.

- A `companies` table represents each tenant.
- A `branches` table belongs to a company (1 company : many branches).
- Every operational table (products, LCs, invoices, expenses, customers, suppliers, etc.) carries both company_id and, where relevant, branch_id.
- A global middleware attaches the requesting user's company_id (and permitted branch_ids) to every request context after JWT verification, and all Sequelize/Prisma queries apply this scope automatically (e.g., via a query middleware / scope helper) to prevent cross-tenant data leakage.

3.2 High-Level Component Diagram (Textual)

- React SPA (Web Client) — role-aware UI, calls REST API over HTTPS.
- Express REST API — organized by module (auth, tenants, lc, purchase, inventory, sales, crm, expense, finance, reports, admin).
- Auth & RBAC Middleware — verifies JWT, resolves company/branch scope and permissions before every controller.
- MySQL Database — single relational store for all tenant and platform data (see §5 Data Model).
- Redis + Job Queue — subscription checks, notifications, scheduled report generation, low-stock alerts.
- Object Storage — product images, LC documents, invoice/challan PDFs.
- Super Admin Portal — a separate protected area of the same app (or a distinct app) for platform management.

3.3 API Design Guidelines

- RESTful, versioned endpoints under /api/v1/. Resource-oriented URLs (e.g., POST /api/v1/lc, GET /api/v1/lc/:id/cost-distribution).
- Consistent JSON response envelope: { success, data, message, meta } for success; { success:false, error: { code, message, details } } for errors.
- Pagination via ?page=&limit=; filtering via query params; sorting via ?sort=field:asc|desc.
- All list endpoints for financial/report data MUST support date-range filters (from, to) and branch filters.
- Idempotency: financial-mutation endpoints (invoice creation, LC cost finalization) should accept an idempotency key to avoid duplicate submissions from double-clicks or retries.
- All monetary amounts transmitted as strings or fixed-point decimals (never raw floats) to avoid rounding errors; store as DECIMAL(15,2) in MySQL.

4. Functional Requirements

Each module below lists: Description, Key Fields/Entities, Business Rules, and Acceptance Criteria. Requirement IDs are provided for traceability.

4.1 Module: SaaS Tenant & Subscription Management [TEN]

Allows a brand-new company to self-register, and allows the Super Admin to manage subscription plans, billing, and tenant lifecycle.

Key Entities / Fields

- Company: id, name, business_type, address, logo, primary_email, phone, currency (default BDT), created_at, status (trial/active/suspended/cancelled).
- SubscriptionPlan: id, name (e.g., Starter/Growth/Enterprise), price, billing_cycle (monthly/yearly), max_branches, max_employees, feature_flags (JSON: LC module on/off, CRM on/off, advanced reports on/off, etc.).
- Subscription: id, company_id, plan_id, status (trial/active/past_due/cancelled), trial_ends_at, current_period_start/end, auto_renew.
- Invoice (Billing): id, company_id, subscription_id, amount, status (paid/unpaid/failed), issued_at, paid_at, payment_reference.

Business Rules

- TEN-01: A new company MUST be created purely by providing an email address (plus company name) — the system sends a verification/OTP email, and upon verification creates the Company, a default Company Owner user, and a default 'Head Office' branch automatically.
- TEN-02: Every new company starts on a configurable free trial (e.g., 14 days) on a default plan; the system MUST block access to paid-tier features once the trial expires until a plan is chosen and paid.
- TEN-03: Feature access (e.g., number of branches, employees, whether advanced reporting is enabled) MUST be enforced server-side based on the company's active plan's feature_flags — not just hidden in the UI.
- TEN-04: Super Admin can manually override a company's plan, extend trials, suspend, or reactivate a tenant.
- TEN-05: When a subscription lapses (payment fails and grace period expires), the company's data remains intact but write access is restricted to billing/read-only until payment resumes.

Acceptance Criteria

- Given a valid email, a user can create a company workspace end-to-end (signup → verify → first login) in under 2 minutes without support intervention.
- A company on the 'Starter' plan attempting to add a branch beyond its plan limit receives a clear upgrade prompt and the action is blocked server-side.

4.2 Module: Authentication, Roles & Access Control [AUTH]

Key Entities / Fields

- User: id, company_id, name, email, phone, password_hash, role_id, status (active/disabled), assigned_branch_ids (array/join table), last_login_at.
- Role: id, company_id (nullable for system default roles), name, is_system_role.

- Permission: id, module, action (create/read/update/delete/approve/export), description.
- RolePermission: role_id, permission_id.

Business Rules

- AUTH-01: Authentication uses JWT access tokens (short-lived, ~15 min) and refresh tokens (longer-lived, httpOnly cookie), with logout invalidating the refresh token.
- AUTH-02: Default system roles (Company Owner, Branch Manager, Sales Staff, Inventory Manager, Accountant, Delivery Coordinator) are seeded per company on creation; the Company Owner MAY create custom roles with custom permission sets.
- AUTH-03: Every API endpoint MUST check both the user's role permissions AND their branch scope (a Branch Manager for Branch A must not read/write Branch B's data unless explicitly granted multi-branch access).
- AUTH-04: Passwords MUST be hashed (bcrypt/argon2); password reset via emailed time-limited token.
- AUTH-05: All permission checks are enforced server-side; the frontend only uses them to control UI visibility.

4.3 Module: Company, Branch & Employee Setup [ORG]

Key Entities / Fields

- Branch: id, company_id, name, address, phone, is_head_office (bool), status.
- Employee (extends User): employee_code, designation, joining_date, assigned_branch_ids, salary (optional, linked to expense module), status.

Business Rules

- ORG-01: A company can create unlimited branches up to its plan limit; each branch has its own inventory, invoice numbering sequence, and expense records, but rolls up into company-wide reports.
- ORG-02: Employees are created under a company and assigned to one or more branches; an employee's visible data (inventory, sales, reports) is restricted to their assigned branch(es) unless their role grants company-wide access.
- ORG-03: Deactivating an employee immediately revokes login access but preserves historical transaction records under their name.

4.4 Module: Supplier Management [SUP]

Key Entities / Fields

- Supplier: id, company_id, name, country, contact_person, phone, email, address, bank_details (for LC), currency_preference (RMB/USD), notes, status.
- SupplierLedger (derived): total purchase value, total paid, outstanding balance, per supplier.

Business Rules

- SUP-01: A supplier record MUST be created/selected before a Purchase Order or LC can be opened against it.
- SUP-02: Supplier list view shows aggregate purchase value and outstanding payable per supplier.

4.5 Module: Letter of Credit (LC) Management [LC] — Core Module

This is the most business-critical module. It governs how import costs become per-unit inventory cost.

Key Entities / Fields

- LC: id, company_id, lc_number (unique per company), issuing_bank, supplier_id, currency (RMB/USD), lc_value (foreign currency), exchange_rate_to_bdt, lc_opening_cost (BDT), status (draft/opened/goods_in_transit/received/cost_finalized/closed), opened_date, expected_arrival_date, actual_arrival_date, notes, attached_documents.
- LCCostItem: id, lc_id, cost_type (enum: lc_opening_cost, freight, port_cost, cnf_cost, transport_cost, unloading_cost, delivery_cost, customs_duty, other), amount_bdt, description, entry_date, entered_by.
- LCProductLine: id, lc_id, product_id/variation_id, quantity, unit_weight (if weight-based distribution used), unit_price_foreign_currency, line_value_foreign_currency.
- LandedCostResult (derived/stored snapshot): lc_id, product_variation_id, base_cost_bdt (converted from FX), distributed_extra_cost_bdt, final_landed_unit_cost_bdt, calculation_basis (weight/quantity), computed_at.

Workflow / Business Rules

- LC-01: Creating an LC requires: supplier, LC number, issuing bank, currency, LC value, and at least one product line (product/variation, quantity, unit price in foreign currency).
- LC-02: The system MUST support recording ALL of the following cost types against an LC, matching the client's cost sheet: LC Opening Cost, Product Purchase Value, Freight, Port Cost, CNF Cost, Transport Cost, Unloading Cost, Delivery Cost, Staff Salary (if allocated to a shipment), Entertainment (if allocated), Customs Duty, and a generic 'Other Charges' category — each as a separate LCCostItem row so every charge is individually auditable.
- LC-03: Users MUST enter or update the exchange rate (RMB→BDT or USD→BDT) for the LC before cost finalization; the base product cost in BDT = quantity × unit_price_foreign_currency × exchange_rate.
- LC-04: Cost Distribution Engine — once all cost items are entered and the LC is marked 'ready to finalize', the system computes total_extra_cost_bdt = sum of all LCCostItem amounts EXCLUDING the base product purchase value. This total is distributed across every unit in the LC's product lines using the user-selected basis: (a) Weight basis: each unit's share = total_extra_cost_bdt × (unit_weight × quantity_of_that_line) / (sum of weight×quantity across all lines); or (b) Quantity/piece basis: each unit's share = total_extra_cost_bdt × quantity_of_that_line / (total quantity across all lines).
- LC-05: final_landed_unit_cost_bdt (per SKU/variation) = base_cost_per_unit_bdt (from FX conversion) + distributed_extra_cost_per_unit_bdt. This value becomes the inventory cost basis when goods are received into stock.
- LC-06: The system MUST retain the full calculation trail (all cost items, the basis used, and per-line results) so any landed cost can be explained/audited later — never store only the final number.
- LC-07: An LC can be edited (cost items added/corrected) even after initial receipt, but any correction MUST trigger a recalculation and create a new LandedCostResult snapshot with a timestamp and reason, without silently overwriting historical inventory valuations already sold.
- LC-08: A single product/SKU MAY exist across multiple LCs (multiple batches) simultaneously; inventory MUST track stock by LC/batch internally (for accurate costing and profit calculation, using FIFO by default) while presenting a single, correctly aggregated total-stock view to the user.
- LC-09: The Delivery Charge module (§4.10) MUST allow entering a delivery cost against a specific LC number, which also feeds into that LC's LCCostItem set for costing accuracy.

- LC-10: LC status progresses: Draft → Opened → Goods in Transit → Received → Cost Finalized → Closed. Only 'Received' or later LCs can have inventory posted from them.

Acceptance Criteria

- Given an LC with 3 cost items and 2 product lines with different weights, selecting 'weight basis' produces per-unit landed costs whose extra-cost portion is proportional to each line's total weight, and the sum of all distributed extra cost across all units exactly equals the entered total extra cost (± rounding tolerance of the smallest currency unit).
- Switching the distribution basis from weight to quantity for the same LC (before finalization) recalculates and shows both results for comparison.
- A user can open the LC detail page and see, for any product, exactly which cost items and what proportion contributed to its final landed cost.

4.6 Module: Purchase Order Management [PO]

Key Entities / Fields

- PurchaseOrder: id, company_id, branch_id, supplier_id, lc_id (nullable if non-LC purchase is allowed), order_date, expected_delivery_date, status (pending/partially_received/received/cancelled).
- PurchaseOrderLine: id, po_id, product_variation_id, quantity, unit_price_foreign_currency or unit_price_bdt.

Business Rules

- PO-01: A Purchase Order is normally created against an LC (for imported goods) but the system SHOULD also support non-LC local purchases (direct BDT purchase from a local supplier) for flexibility.
- PO-02: Receiving a Purchase Order's goods (fully or partially) triggers Goods Receipt into inventory at the applicable unit cost (landed cost for LC-based POs, or entered cost for local POs).
- PO-03: An order can be cancelled before receipt; a partially received order can only cancel the unreceived remainder.

4.7 Module: Product & Inventory Management [INV]

Key Entities / Fields

- Product: id, company_id, name, category, description, unit_of_measure (pcs/kg/set), image, status.
- ProductVariation: id, product_id, variation_name (e.g., '6-inch profile', '8-inch profile', 'clear glass 5mm'), sku, unit_weight, reorder_level.
- InventoryBatch: id, product_variation_id, branch_id, lc_id (nullable), source_po_id, quantity_received, quantity_available, unit_cost_bdt, received_date.
- StockLedgerEntry: id, product_variation_id, branch_id, batch_id, transaction_type (purchase_in/sale_out/return_in/adjustment/transfer), quantity, balance_after, reference_id, created_at.

Business Rules

- INV-01: Each product supports multiple variations, each with its own price and stock; e.g., an aluminium product with 3 variations must show independent pricing and stock per variation.
- INV-02: Inventory MUST track stock at the batch (LC/PO) level internally for costing (FIFO issue by default for cost-of-goods-sold calculation) but display a single accurate aggregated total per variation per branch to end users.

- INV-03: The system MUST support: Total Stock (company-wide), Item-wise/Variation-wise Stock, Branch-wise Balance Stock, and low-stock alerts based on reorder_level.
- INV-04: Every stock-affecting action (sale, purchase receipt, return, manual adjustment, inter-branch transfer) MUST create a StockLedgerEntry row — stock quantities are never edited directly without a ledger trail.
- INV-05: Stock adjustments (damage, count correction) require a reason code and are visible in reports separately from normal sales/purchase movement.

4.8 Module: Customer Management (CRM) [CRM]

Key Entities / Fields

- Customer: id, company_id, name, phone, email, address, customer_type (retail/wholesale/dealer), credit_limit, opening_balance, current_balance (derived), status.
- CustomerLedgerEntry: id, customer_id, transaction_type (invoice/payment/return/credit_note), amount, balance_after, reference_id, date.

Business Rules

- CRM-01: Every customer has a configurable credit_limit; a credit invoice that would push the customer's outstanding balance above their credit_limit MUST be blocked (or require Manager/Owner override) at point of sale.
- CRM-02: opening_balance can be set once at customer creation (for migrating existing dues) and thereafter only changes through ledger transactions.
- CRM-03: The customer detail page shows full ledger history, current outstanding, and product-purchase history (for the 'customer product-wise' report requested by the client).

4.9 Module: Sales & Invoicing [SLS]

Key Entities / Fields

- Invoice: id, company_id, branch_id, invoice_number (branch-scoped sequence), invoice_type (cash/credit), customer_id (nullable for walk-in cash sales), salesman_id, invoice_date, subtotal, discount, tax, total, paid_amount, due_amount, status (draft/confirmed/cancelled/returned).
- InvoiceLine: id, invoice_id, product_variation_id, batch_id (for costing), quantity, unit_price, line_total, unit_cost_bdt (snapshotted for profit calc).
- Challan: id, invoice_id, challan_number, delivery_address, delivery_person, delivery_date, status (pending/delivered).
- MoneyReceipt: id, company_id, customer_id, invoice_id (nullable, can be a general payment against ledger), amount, payment_method (cash/bank/mobile banking), receipt_number, date.
- CreditNote: id, company_id, customer_id, related_invoice_id, amount, reason, date.
- SalesReturn: id, invoice_id, invoice_line_id, quantity_returned, reason, date, restock (bool).

Business Rules

- SLS-01: Two invoice types are supported — Cash Invoice (fully paid at point of sale) and Credit Invoice (partial/no payment, tracked against customer credit_limit and ledger).
- SLS-02: Every confirmed invoice MUST automatically deduct stock (FIFO across available batches for that variation/branch) and create corresponding StockLedgerEntry rows.

- SLS-03: Every confirmed invoice generates a printable Challan (delivery note) as a distinct document from the invoice, containing items and quantities but optionally hiding prices, for the delivery person.
- SLS-04: Payments against a credit invoice are recorded as MoneyReceipts, which update the invoice's paid_amount/due_amount and the customer's ledger balance.
- SLS-05: Invoice Cancel: allowed only before delivery/payment is finalized (configurable window) and MUST reverse stock deduction and any ledger postings.
- SLS-06: Sales Return: supports Cash Invoice Return and Credit Invoice Return, each restocking (optional) and, for credit, issuing a Credit Note that reduces the customer's outstanding balance.
- SLS-07: Order Cancel / Sales Order Cancel: supported at the pre-invoice quotation/order stage, distinct from invoice cancellation.
- SLS-08: Every invoice/challan tracks its print status (e.g., 'not printed' / 'printed' / 'reprinted N times') to support the client's requested 'Invoice Print Status' report.
- SLS-09: Salesman-wise sales MUST be trackable via the salesman_id on each invoice for the Salesman-wise Sales Report.
- SLS-10: Profit per invoice line = (unit_price − unit_cost_bdt) × quantity, using the snapshotted landed cost at time of sale — this feeds the Profit & Loss reports.

4.10 Module: Delivery & Challan Management [DLV]

Business Rules

- DLV-01: A dedicated 'Delivery Charge Entry' screen allows recording a delivery cost tied to a specific LC number (in addition to per-invoice challans), matching the client's explicit requirement for LC-linked delivery charge tracking.
- DLV-02: Challans can be marked delivered/undelivered, and support reprinting with an audit log of reprint count and user.
- DLV-03: A 'Total Delivery Charge' report aggregates delivery cost by LC, by branch, and by date range.

4.11 Module: Daily Expense Management [EXP]

Key Entities / Fields

- ExpenseCategory: id, company_id, name (e.g., Office Rent, Warehouse Rent, Staff Salary, Entertainment, Utility, Delivery Charge, Customs Duty, Other), is_system_default.
- Expense: id, company_id, branch_id, category_id, amount, lc_id (nullable, for LC-linked charges), description, date, entered_by, attachment.

Business Rules

- EXP-01: Default expense categories are seeded to match the client's cost sheet: Office Rent, Warehouse Rent, Staff Salary, Entertainment, plus any company-defined custom categories.
- EXP-02: Expenses MAY optionally be linked to an LC (e.g., a portion of staff salary or entertainment allocated to handling a specific shipment), feeding into that LC's cost distribution if flagged as such; otherwise they are purely operating expenses for the P&L.
- EXP-03: A Daily Expense Summary and a Daily Cash Closing report are generated automatically, aggregating all expenses and cash sales/receipts for the day, per branch.

4.12 Module: Finance & Cash Management [FIN]

Business Rules

- FIN-01: Daily Cash Closing: for each branch, per day, the system computes Opening Cash + Cash Sales + Cash Receipts − Cash Expenses − Cash Payouts = Closing Cash, allowing a manager to confirm/reconcile against physically counted cash.
- FIN-02: Profit & Loss (Daily/Monthly): Revenue (from invoices) − Cost of Goods Sold (from snapshotted landed costs) − Operating Expenses (from EXP module) = Net Profit/Loss, viewable by day, month, and branch.
- FIN-03: Accounts Payable: outstanding amounts owed to suppliers (derived from LC value and any recorded supplier payments) are tracked, even though the LC payment itself happens at the bank.

4.13 Module: Reporting & Analytics [RPT]

All reports below MUST support date-range and branch filters, and export to PDF/Excel. This list consolidates and formalizes the reports explicitly requested by the client.

Report	Description
Daily Transaction Summary	All cash/credit invoices, receipts, and expenses for a given day.
Daily/Monthly Sales Report	Total sales, by branch, by salesman, by product.
Total Purchase Report	Purchases by supplier, by LC, by date range.
Total Sales / Profit & Loss	Revenue, COGS, expenses, net profit — daily/monthly.
Total Inventory / Branch-wise Inventory	Stock on hand, valued at landed cost, per branch and company-wide.
Item-wise / Balance Stock	Per-SKU quantity available, reorder alerts.
Customer Outstanding / Customer Ledger	Per-customer balances and full transaction history.
Supplier Data / Supplier List	Purchase volume and outstanding per supplier.
Salesman-wise Sales Report	Sales performance per salesman.
Total Return Report	All sales returns, by product, by customer, by reason.
Total Delivery Charge Report	Delivery costs by LC, branch, date range.
Invoice Print Status Report	Which invoices/challans are printed, unprinted, or reprinted.
Top Selling Item Report	Best-selling products/variations by quantity and revenue, filterable by customer.
Customer Product-wise Report	What each customer has historically purchased.
LC Cost & Landed Cost Report	Full cost breakdown and landed cost derivation per LC.
Daily Cash Closing Report	Reconciled daily cash position per branch.

4.14 Module: Notifications & Audit Trail [AUD]

- AUD-01: Every create/update/delete on financially sensitive entities (LC, invoice, expense, cost item, customer credit limit) MUST write an audit-log entry: who, what, before/after values, timestamp.
- AUD-02: In-app and/or email notifications for: low stock, LC arrival due date approaching, credit invoice overdue, subscription renewal due/failed.

5. Data Model Overview

The following table summarizes the core entities and their primary relationships. Full column-level schemas should be derived from the field lists in §4; this table is the relational skeleton Claude Code should implement first as MySQL tables/Sequelize (or Prisma) models.

Entity	Key Relationships
Company	1—* Branch, User, Supplier, Customer, Product, LC, Invoice, Expense, Subscription
Branch	Company *—1; 1—* Employee-assignment, InventoryBatch, Invoice, Expense
User / Employee	Company *—1; Role *—1; *—* Branch (assignment)
Role / Permission	Role *—* Permission; Role 1—* User
Supplier	Company *—1; 1—* LC, PurchaseOrder
LC	Supplier *—1; 1—* LCCostItem, LCProductLine, PurchaseOrder; 1—* LandedCostResult
PurchaseOrder	LC *—1 (nullable); Supplier *—1; 1—* PurchaseOrderLine; 1—* InventoryBatch (on receipt)
Product / ProductVariation	Company *—1; 1—* InventoryBatch, InvoiceLine, LCProductLine
InventoryBatch	ProductVariation *—1; Branch *—1; LC/PO *—1; 1—* StockLedgerEntry
Customer	Company *—1; 1—* Invoice, CustomerLedgerEntry, MoneyReceipt, CreditNote
Invoice	Customer *—1 (nullable); Branch *—1; 1—* InvoiceLine, Challan; *—1 Salesman(User)
Expense	Company *—1; Branch *—1; ExpenseCategory *—1; LC *—1 (nullable)
Subscription / SubscriptionPlan	Company 1—1 (active) Subscription *—1 SubscriptionPlan

6. Non-Functional Requirements

6.1 Security

- NFR-SEC-01: Strict tenant data isolation — no query may ever return data across company_id boundaries; enforced via a mandatory server-side scoping layer, not client-supplied filters.
- NFR-SEC-02: All traffic over HTTPS/TLS; passwords hashed with bcrypt/argon2; JWT secrets and DB credentials in environment variables / secrets manager, never in source control.
- NFR-SEC-03: Role- and branch-based authorization enforced on every API endpoint, server-side.
- NFR-SEC-04: Full audit trail for financial and access-control changes (see AUD-01).
- NFR-SEC-05: Rate limiting and brute-force protection on authentication endpoints; account lockout after repeated failed logins.
- NFR-SEC-06: Regular automated encrypted database backups with tested restore procedures; point-in-time recovery target of ≤24 hours data loss.
- NFR-SEC-07: Input validation and parameterized queries throughout (ORM-based) to prevent SQL injection; output encoding to prevent XSS.

6.2 Performance & Scalability

- NFR-PERF-01: Core POS actions (create invoice, look up stock) MUST respond in under 1 second under normal load for a single tenant with up to ~50,000 SKUs and ~500,000 transactions/year.
- NFR-PERF-02: Report generation for date ranges up to 1 year MUST complete in under 5 seconds, using appropriate indexing and, if needed, pre-aggregated summary tables for heavy reports.
- NFR-PERF-03: The architecture MUST allow horizontal scaling of the API layer and read-replica scaling of MySQL as tenant count grows.

6.3 Availability & Reliability

- NFR-AVL-01: Target uptime of 99.5% for the production SaaS environment.
- NFR-AVL-02: Financial mutations (invoice, LC cost finalization) MUST be wrapped in database transactions to guarantee consistency (no half-committed stock/ledger states).

6.4 Usability / UI-UX

- NFR-UX-01: The UI MUST be modern, clean, and 'sophisticated' per the client's request — use a professional design system (spacing, typography, and color consistent with the frontend-design guidance used for this build), not default/unstyled component library looks.
- NFR-UX-02: The Sales/POS screen MUST be optimized for speed: keyboard-friendly, minimal clicks to complete a cash sale, product search by name/SKU/barcode-ready.
- NFR-UX-03: Support both English and Bengali labels where practical (at minimum, number/date formats appropriate for Bangladesh; full i18n is a should-have, not a hard blocker for v1.0).
- NFR-UX-04: All data tables (inventory, invoices, ledgers) support search, filter, sort, and pagination.

6.5 Maintainability & Code Quality

- NFR-MNT-01: Modular backend architecture (controllers/services/repositories) so each functional module in §4 maps to a clear code module.
- NFR-MNT-02: Consistent naming, environment-based configuration, and automated migrations for all MySQL schema changes (e.g., Sequelize/Prisma migrations) — no manual, undocumented schema edits.
- NFR-MNT-03: Automated tests for core financial logic, especially the LC cost-distribution engine (LC-04/LC-05), given its business criticality.

6.6 Compliance & Data Retention

- NFR-CMP-01: Financial records (invoices, LC costs, expenses) MUST be retained indefinitely by default (configurable per company) to support audits and tax reporting.
- NFR-CMP-02: On subscription cancellation, tenant data is retained in a read-only/export-only state for a defined grace period (e.g., 90 days) before deletion, with the company able to export all data at any time.

7. Subscription & Billing Model

- Plans are tiered (e.g., Starter, Growth, Enterprise) and differ by: number of branches, number of employees, LC/import module inclusion, advanced reporting, and support level. Exact pricing is a business decision outside this SRS; the system must simply support configurable plans via feature_flags and limits.
- Company Owners can view their current plan, usage against limits (branches/employees used vs. allowed), upgrade/downgrade, and view/download billing invoices from within the app.
- Failed payments trigger a grace period with in-app and email warnings before feature restriction (see TEN-05).
- The Super Admin portal provides a dashboard of all tenants, their plan, MRR (monthly recurring revenue), churn, and trial-to-paid conversion.

8. Worked Example — Landed Cost Calculation

This illustrates the calculation Claude Code must implement (LC-03 through LC-05), using illustrative figures consistent with the client's notes.

Scenario

- LC-2026-0091 opened for Supplier 'Foshan Nanhai Guangjun Aluminium'.
- Two product lines: Line A — Aluminium Profile Type-1, 2,000 pcs, unit weight 0.5 kg, unit price 8 RMB. Line B — Aluminium Profile Type-2, 1,000 pcs, unit weight 0.8 kg, unit price 12 RMB.
- Exchange rate: 1 RMB = 16.50 BDT (entered by user at cost-finalization time).
- Additional cost items entered: LC Opening Cost 15,000 BDT; Freight 40,000 BDT; Port Cost 10,000 BDT; CNF Cost 8,000 BDT; Transport Cost 12,000 BDT; Unloading Cost 5,000 BDT; Customs Duty 60,000 BDT. Total extra cost = 150,000 BDT.

Step 1 — Base cost per unit (from FX)

- Line A base unit cost = 8 RMB × 16.50 = 132.00 BDT/pc.
- Line B base unit cost = 12 RMB × 16.50 = 198.00 BDT/pc.

Step 2 — Distribute extra cost (weight basis)

- Total weight = (2,000 × 0.5) + (1,000 × 0.8) = 1,000 + 800 = 1,800 kg.
- Line A share of extra cost = 150,000 × (1,000 / 1,800) = 83,333.33 BDT → per unit = 83,333.33 / 2,000 = 41.67 BDT/pc.
- Line B share of extra cost = 150,000 × (800 / 1,800) = 66,666.67 BDT → per unit = 66,666.67 / 1,000 = 66.67 BDT/pc.

Step 3 — Final landed unit cost

- Line A final landed cost = 132.00 + 41.67 = 173.67 BDT/pc.
- Line B final landed cost = 198.00 + 66.67 = 264.67 BDT/pc.

If 'quantity basis' were selected instead, the 150,000 BDT would be split by piece count (2,000 vs 1,000 of a 3,000 total) rather than by weight — the system must support both and let the user choose per LC, as specified in LC-04.

9. Permission Matrix (Summary)

Full = create/read/update/delete. View = read-only. None = no access. This is a starting default; the Company Owner can customize via custom roles (AUTH-02).

Module	Owner	Branch Mgr	Sales Staff	Inventory Mgr	Accountant
LC Management	Full	View	None	View	Full
Purchase Orders	Full	Full	None	Full	View
Inventory	Full	Full	View	Full	View
Sales / Invoicing	Full	Full	Full	None	View
Customers (CRM)	Full	Full	View	None	View
Expenses	Full	Full	None	None	Full
Finance / P&L	Full	View	None	None	Full
Reports	Full	Full (own branch)	Own sales only	Stock reports	Financial reports
Users & Roles	Full	None	None	None	None
Subscription/Billing	Full	None	None	None	None

10. Suggested Development Phases (for Claude Code)

This phasing is a suggestion to sequence implementation logically; Claude Code may adapt as needed but should not defer the LC costing engine, as later modules depend on it.

11. Phase 1 — Foundation: Multi-tenant DB schema, company signup/onboarding, auth & RBAC, branch/employee setup, Super Admin skeleton.
12. Phase 2 — Master Data: Supplier management, Product/Variation management, Customer (CRM) management.
13. Phase 3 — Core LC & Purchasing: LC module (including the cost-distribution engine and worked-example logic), Purchase Orders, Goods Receipt into Inventory.
14. Phase 4 — Sales & POS: Cash/Credit invoicing, Challan printing, Money Receipts, Returns, Credit Notes, Order/Invoice cancellation.
15. Phase 5 — Finance & Expenses: Expense entry (incl. LC-linked delivery charges), Daily Cash Closing, Profit & Loss.
16. Phase 6 — Reporting Suite: All reports listed in §4.13, with export to PDF/Excel.
17. Phase 7 — SaaS Commercialization: Subscription plans, billing, feature gating, Super Admin analytics dashboard.
18. Phase 8 — Polish: UI refinement, i18n (English/Bengali), notifications, audit-log UI, performance tuning.

Appendix A — Suggested Menu / Module Structure

- Dashboard
- LC Management → LC List, New LC, Cost Entry, Cost Distribution & Landed Cost, LC-linked Delivery Charges
- Purchasing → Purchase Orders, Goods Receipt
- Inventory → Products & Variations, Stock by Branch, Item-wise Stock, Stock Adjustments, Low Stock Alerts
- Sales (POS) → New Invoice (Cash / credit mode selection), Invoice List, Challans, Money Receipts, Returns, Credit Notes, and advance payment , invoice or challan print
- Product Returns → Product returns, return invoices, cash returns and need admin aproval
- Customers (CRM) → Customer List, Customer Ledger, Credit Limits, opening balance (credit or advance)
- Suppliers → Supplier List, Supplier Ledger
- Expenses → Daily Expense Entry, Expense Categories
- Finance → Daily Cash Closing, Profit & Loss, Accounts Payable
- Reports → (full list per §4.13)
- Organization → Branches, Employees, Roles & Permissions
- Settings → Company Profile, Subscription & Billing, Notification Preferences
- Super Admin (separate area) → Tenants, Plans, Billing, Platform Analytics

Appendix B — Source Notes

This SRS formalizes handwritten process notes provided by the client (GK Interior), covering: (1) the LC cost sheet — LC opening cost, product purchase, freight, port cost, CNF cost, transport cost, unloading, delivery, staff salary, entertainment, and customs duty; (2) the two invoice/document types for cash & sell — invoice, challan, and money receipt; and (3) the full L/C Management feature checklist — purchase order/product value, sales/invoice/challan, profit & loss, daily expenses, stock management, customer data, monthly sales/profit/expense reports, supplier data, cash/credit invoice, money receipt, total/item-wise/balance stock, supplier & customer lists, customer limits & opening balance, invoice/order cancellation, credit notes, invoice print status, and daily sales/top-selling-item reporting. Every one of these items has been mapped to a specific module and requirement ID in §4 above.